Waylar OMS — External API v1.33.0

Base URL: `https://oms.waylar.net/api/v1`

Authentication

Every request must include:

```
X-API-Key: wlr_<key>
```

Get a key from **Settings** → **API Keys** in the OMS dashboard.

Products

List Products

```
GET /api/v1/products
```

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)

Query Parameters

Param	Type	Description
`q`	string	Search by SKU or product name
`page`	number	Page number (default: 1)
`limit`	number	Items per page, max 100 (default: 20)
`dir`	`asc` | `desc`	Sort by id (default: desc)
`active`	`0` | `1`	Filter by active status

Example Request

```
curl "https://oms.waylar.net/api/v1/products?q=shirt&limit=10" \
  -H "X-API-Key: wlr_abc123..."
```

Response `200`


```
{
  "data": [
    {
      "id": 42,
      "sku": "SHIRT-WHITE-M",
      "name": "White T-Shirt",
      "price": 299,
      "cost": 120,
      "stock": 50,
      "active": 1,
      "isDigital": 0,
      "margin": 60
    }
  ],
  "total": 1,
  "page": 1,
  "limit": 10,
  "pages": 1
}
```

Create Product

```
POST /api/v1/products
```

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)
`Content-Type`	`application/json`

Request Body

Field	Type	Required	Description
`name`	string	✅	Product name
`sku`	string	✅	Product SKU (unique per org)
`price`	number	✅	Selling price

Field	Type	Required	Description
`cost`	number		Cost price (default: 0)
`stock`	number		Stock quantity (default: 0)
`description`	string		Product description
`imageUrl`	string		Product image URL
`expiry`	string		Expiry date, `YYYY-MM-DD`
`weightKg`	number		Weight in kg, optional — used when handing the order off to a courier
`active`	boolean		Enable/disable product (default: true)
`isDigital`	boolean		Digital product — no shipping needed; stock is cut immediately on payment instead of waiting on ship (default: false)

> **Note:** `categoryId`, `warehouseId`, `reorder`, `leadTimeHours`, `channels` exist as fields on the product but this endpoint does not accept them on create yet (they'll always get their default value) — set those from the OMS dashboard instead.

Example Request

```
curl -X POST https://oms.waylar.net/api/v1/products \
  -H "X-API-Key: wlr_abc123..." \
  -H "Content-Type: application/json" \
  -d '{
    "name": "White T-Shirt",
    "sku": "SHIRT-WHITE-M",
    "price": 299,
    "cost": 120,
    "stock": 50
  }'
```

Response `201 Created`

```
{
  "id": 42,
  "sku": "SHIRT-WHITE-M",
  "name": "White T-Shirt",
  "description": "",
  "categoryId": null,
  "warehouseId": null,
  "cost": 120,
  "price": 299,
  "stock": 50,
  "reorder": 10,
  "leadTimeHours": 0,
  "imageUrl": "",
  "expiry": "",
  "weightKg": null,
  "channels": [],
  "active": 1,
  "isDigital": 0,
  "margin": 60,
  "createdAt": "2026-07-20T10:00:00.000Z"
}
```

Error Responses

Status	Condition
`401`	API key invalid or revoked
`400`	Missing required field, or duplicate SKU

```
{ "error": "name, sku, price are required" }
```

```
{ "error": "duplicate key value violates unique constraint
\"products_org_id_sku_unique\"" }
```

Update Stock

```
PATCH /api/v1/products/{sku}/stock
```

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)
`Content-Type`	`application/json`

Path Parameters

Param	Type	Description
`sku`	string	Product SKU

Request Body

Field	Type	Required	Description
`stock`	number	✅	New stock quantity (absolute value, not a delta)

Example Request

```
curl -X PATCH https://oms.waylar.net/api/v1/products/SHIRT-WHITE-M/stock \
  -H "X-API-Key: wlr_abc123..." \
  -H "Content-Type: application/json" \
  -d '{ "stock": 80 }'
```

Response `200`

```
{
  "id": 42,
  "sku": "SHIRT-WHITE-M",
  "name": "White T-Shirt",
  "stock": 80,
  "price": 299,
  "cost": 120,
  "active": 1,
  "isDigital": 0,
  "margin": 60
}
```

Error Responses

Status	Condition
`401`	API key invalid
`400`	Missing `stock`, or not a number
`404`	SKU not found

```
{ "error": "stock (number) is required" }
```

Customers

List Customers

```
GET /api/v1/customers
```

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)

Query Parameters

Param	Type	Description
`q`	string	Search by code, name, phone, or email
`page`	number	Page number (default: 1)
`limit`	number	Items per page, max 100 (default: 20)
`dir`	`asc` | `desc`	Sort by id (default: desc)

Example Request

```
curl "https://oms.waylar.net/api/v1/customers?q=somchai&limit=10" \
  -H "X-API-Key: wlr_abc123..."
```

Response `200`

```
{
  "data": [
    {
      "id": 52,
      "code": "C37725",
      "name": "Somchai Jaidee",
      "phone": "0812345678",
      "email": "somchai@example.com",
      "tax_id": "",
      "address": "",
      "points": 0,
      "stage": "Lead",
      "created_at": "2026-08-20T10:00:00.000Z"
    }
  ],
  "total": 1,
  "page": 1,
  "limit": 10,
  "pages": 1
}
```


Soft-deleted customers (merged into another record) are excluded from this list.

Error Responses

Status	Condition
`401`	API key invalid

Create Customer

```
POST /api/v1/customers
```

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)
`Content-Type`	`application/json`

Request Body

Field	Type	Required	Description
`name`	string	✅	Customer name

Field	Type	Required	Description
`code`	string		Customer code (auto-generated if omitted)
`phone`	string		Phone number
`email`	string		Email
`tax_id`	string		Tax ID / national ID
`address`	string		Address

Example Request

```
curl -X POST https://oms.waylar.net/api/v1/customers \
  -H "X-API-Key: wlr_abc123..." \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Somchai Jaidee",
    "phone": "0812345678",
    "email": "somchai@example.com"
  }'
```

Response `201 Created`

```
{
  "id": 52,
  "code": "C889012",
  "name": "Somchai Jaidee",
  "phone": "0812345678",
  "email": "somchai@example.com",
  "tax_id": "",
  "address": "",
  "points": 0,
  "stage": "Lead",
  "created_at": "2026-08-26T10:00:00.000Z"
}
```

Error Responses

Status	Condition
`401`	API key invalid
`400`	Missing `name`

Status	Condition
`400`	`tax_id` invalid, or already used by another customer in your org (see Update Customer Bank Account)

```
{ "error": "name is required" }
```

Update Customer Bank Account

```
PATCH /api/v1/customers/{id}
```

Sets/updates a customer's receiving bank account — use this when the customer will be the seller in a **buyback order** (see Create a Buyback Order). Set once, reused across every buyback order for that customer. This field set matches exactly what the payout provider needs to actually pay someone — if any field is missing, no buyback order can be created for that customer at all.

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)
`Content-Type`	`application/json`

Path Parameters

Param	Type	Description
`id`	number	Customer id

Request Body (send only the fields you want to change — omitted fields are left untouched)

Field	Type	Description
`tax_id`	string	The customer's 13-digit tax id (national ID for an individual) — **required for buyback orders** (some payout routes only)
`email`	string	Recipient's email — required by some payout routes; settable here or at customer creation

Field	Type	Description
`bank_account_type`	string	`"account"` or `"promptpay"`
`bank_code`	string	Bank code (e.g. `"014"` = SCB)
`bank_account_no`	string	Account number / PromptPay number
`bank_account_name`	string	Account name
`given_name`	string	Given name (can be used instead of `bank_account_name`)
`surname`	string	Surname
`address_country`	string	ISO country code, e.g. `"TH"`
`address_city`	string	City / province — **required for buyback orders**
`address_street`	string	Street address — **required for buyback orders**

`bank_account_no` **and** (`bank_account_name` or `given_name` + `surname`) must both be set before a buyback order can be created for this customer.

Always fill in `email`. The payout route Waylar uses can change at any time and some routes require the recipient's email address; without it, a buyback order for that customer either cannot be created (`400`) or cannot be paid out at verify time (`409`). This is a Waylar-side setting with no endpoint for a partner to read or change.

`tax_id` is required for buyback orders (some payout routes only) — this endpoint requires the customer's 13-digit `tax_id` the same way it requires `email`, since the payment voucher filed is under the recipient customer's own tax id, not the merchant's. A customer without a valid 13-digit `tax_id` blocks `order_type: "buy"` order creation (`400`).

`tax_id` is validated on write (here and on `POST /customers`), returning `400` when:

- it is not 13 digits with a correct check digit (`"เลขผู้เสียภาษีไม่ถูกต้อง ..."`) — non-digits such as dashes are stripped first and the digits-only form is what is stored
- another customer in your org already has it (`"เลขผู้เสียภาษีนี้ถูกใช้กับลูกค้ารายอื่นในร้านแล้ว"`) — a payout voucher is filed under this number, so it belongs to one customer only. The other customer is deliberately not named. The same number in a *different* org is fine.

Sending `""` clears it.

Example Request

```
curl -X PATCH https://oms.waylar.net/api/v1/customers/52 \
  -H "X-API-Key: wlr_abc123..." \
  -H "Content-Type: application/json" \
  -d '{
    "tax_id": "1234567890121",
    "bank_account_type": "account",
    "bank_code": "014",
    "bank_account_no": "1234567890",
    "given_name": "Somchai",
    "surname": "Jaidee",
    "address_country": "TH",
    "address_city": "Bangkok",
    "address_street": "123 Sukhumvit Rd."
  }'
```

Response `200`

```
{
  "id": 52,
  "code": "C37725",
  "name": "Somchai Jaidee",
  "tax_id": "1234567890121",
  "bank_account_type": "account",
  "bank_code": "014",
  "bank_account_no": "1234567890",
  "bank_account_name": "",
  "given_name": "Somchai",
  "surname": "Jaidee",
  "address_country": "TH",
  "address_city": "Bangkok",
  "address_street": "123 Sukhumvit Rd.",
  "bank_book_url": "",
  "id_copy_url": ""
}
```

`bank_book_url` / `id_copy_url` are internal GCS object paths, not browsable links — empty until a document is uploaded via the endpoint below, the only one that returns something viewable (`view_url`).

Error Responses

Status	Condition
`401`	API key invalid
`404`	Customer id not found
`400`	No recognized fields in the body
`400`	`tax_id` invalid, or already used by another customer in your org

---

Upload Customer Document

```
POST /api/v1/customers/{id}/documents
```

Uploads a supporting document for the customer's payout bank account — a **bank book photo** (`bank_book` , clearly showing the account number/name) or an **ID card copy** (`id_copy`). Re-uploading replaces the previous file (the old GCS object is simply orphaned, not deleted).

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)
`Content-Type`	`multipart/form-data` (not JSON)

Path Parameters

Param	Type	Description
`id`	number	customer id

Form Fields

Field	Type	Required	Description
`type`	string	✅	`"bank_book"` or `"id_copy"`
`file`	file	✅	PDF or image (png/jpg/webp), 16 MB max

Example Request

```
curl -X POST https://oms.waylar.net/api/v1/customers/52/documents \
  -H "X-API-Key: wlr_abc123..." \
  -F "type=bank_book" \
  -F "file=@bank-book-page1.jpg"
```

Response `200`

```
{ "ok": true, "type": "bank_book", "view_url": "https://storage.googleapis.com/...
(signed, 10 min)" }
```

`view_url` is a signed URL valid for 10 minutes — only meant to confirm the upload right away. There's no separate endpoint to re-request a view_url later; upload again if you need a fresh link.

Error Responses

Status	Condition
`401`	API key invalid
`404`	Customer id not found
`400`	`type` is not `bank_book` / `id_copy`
`400`	No file, or file over 16 MB
`400`	File is not a supported PDF/image type

Orders

List Orders

```
GET /api/v1/orders
```

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)

Query Parameters

Param	Type	Description
`page`	number	Page number (default: 1)

Param	Type	Description
`limit`	number	Items per page, max 100 (default: 20)
`dir`	`asc` | `desc`	Sort direction (default: desc) — by id, or by `paid_at` with `sort=paid_at`
`status`	string	Filter by order status — see the enum below
`payment_status`	string	Filter by `unpaid` | `pending` | `paid`
`sort`	`id` | `paid_at`	`id` (default) as before. `paid_at`: **paid orders only**, by payment time then id, paged by `cursor` (v1.33.0)
`cursor`	string	With `sort=paid_at` only: the `next_cursor` of the previous page. Omit for the first page

Example Request

```
curl "https://oms.waylar.net/api/v1/orders?payment_status=pending&limit=10" \
  -H "X-API-Key: wlr_abc123..."
```

Response `200`

```
{
  "data": [
    {
      "id": 101,
      "code": "SO-12345678",
      "status": "Pending",
      "channel": "LINE",
      "payment_status": "pending",
      "total": 748,
      "paid": 0,
      "paid_amount": 0,
      "is_paid": false,
      "payment_reference_id": null,
      "paid_at": null,
      "payer_name": null,
      "note": "ส่งด่วน",
      "deliver_at": "",
      "created_at": "2026-08-20T10:00:00.000Z"
    }
  ],
  "total": 1,
  "page": 1,
  "limit": 10,
  "pages": 1
}
```


`note` (v1.33.0) — the order's note as created; `null` when empty. Also on `GET /orders/{code}/status`.

Paging by payment time (v1.33.0). `sort=paid_at` lists only paid orders, ordered by `paid_at` then `id` (`dir` applies), and pages with a cursor instead of `page`:

```
curl "https://oms.waylar.net/api/v1/orders?sort=paid_at&dir=desc&limit=100" -H "X-API-
Key: wlr_abc123..."
curl "https://oms.waylar.net/api/v1/orders?sort=paid_at&dir=desc&limit=100&cursor=
<next_cursor>" -H "X-API-Key: wlr_abc123..."
```


```
{ "data": [ … ], "limit": 100, "next_cursor":
"WyIyMDI2LTEwLTI2VDAzOjA0OjA1LjAwMFoiLDEyMzQ1Nl0" }
```


`next_cursor` is `null` on the last page. Treat it as opaque, and keep the same `dir` for the whole walk. Rows don't shift between pages when new orders are paid. `paid_at` is the gateway's payment time, so an order confirmed late can carry a `paid_at` earlier than ones you have already read: for a complete reconciliation walk with `dir=desc` from the newest, and overlap the previous run by a few minutes. This mode has no `total` / `page` / `pages`. Other filters (`status` , `payment_status` , `paid_from` , `paid_to`) combine with it. Without `sort` / `cursor` the response is exactly as before.

`payer_name` — the name on the account the customer paid from as the gateway reported it. `null` when unpaid or when the gateway sends no payer. Same field as `transfer.payer_name` on `GET /orders/{code}/status` — call that endpoint if you also need `pv_no` or the net payout amount.

`paid` here is a **number** (amount paid so far) — different from `paid` on `GET /orders/{code}/status`, which is a **boolean**. Use `paid_amount` (number) and `is_paid` (boolean) instead if that ambiguity matters to you — the original `paid` stays for backward compat.

`status` (order status) is one of, for sale orders: `"Pending"` → `"Shipping"` → `"Delivered"`, or `"Cancelled"` at any point (buyback orders have their own separate enum: `"Pending"` → `"Shipping"` → `"Received"`, or `"Cancelled"`). The cancellation reason (`cancel_reason`) is never exposed via this external API at all — only visible in the OMS dashboard.

This list endpoint's response has no `order_type` field (nor does `GET /orders/{code}/status` — only `POST /orders`'s create response includes it). The only way to tell a sale order from a buyback one here is the `code` prefix (`SO-` = sale, `BO-` = buyback), or whether `buy_verify_status` is non-null.

Create Order

```
POST /api/v1/orders
```

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)
`Content-Type`	`application/json`

Request Body

Field	Type	Required	Description
`items`	array	✅	Line items (at least 1 required)
`items[].sku`	string	✅	Product SKU
`items[].qty`	number	✅	Quantity
`items[].price`	number		Unit price override (default: the product's list price)

Field	Type	Required	Description
`customer_id`	number	⚠️	Customer id —**required for some orgs**, see note below
`channel`	string		Channel, e.g. `"Shopee"`, `"LINE"` (default: `"Shopee"`)
`note`	string		Note
`deliver_at`	string		Requested delivery date, ISO 8601, e.g. `"2026-07-25T10:00:00Z"`
`utm_source`	string		Traffic source
`utm_campaign`	string		Campaign
`order_type`	string		`"sale"` (default) or `"buy"` — see Create a Buyback Order below if you're creating a buyback order

⚠️ **`customer_id`**: only required if your org has "require customer_id" turned on (Waylar enables this per org from the backoffice, only after telling you first) — most orgs keep this optional. If your org has it on and you don't send one, you'll get `400 customer_id required`. Why it's required: the payer-name review (comparing who paid against the order's customer — see `transfer.review_status` in Check Payment Status) needs a customer to compare against. (This rule applies only to `order_type: "sale"` — buyback orders have their own separate `customer_id` rule, see the next section.)

Example Request

```
curl -X POST https://oms.waylar.net/api/v1/orders \
  -H "X-API-Key: wlr_abc123..." \
  -H "Content-Type: application/json" \
  -d '{
    "items": [
      { "sku": "SHIRT-WHITE-M", "qty": 2 },
      { "sku": "HAT-BLACK",     "qty": 1, "price": 150 }
    ],
    "channel": "LINE",
    "note": "Rush delivery"
  }'
```

Response `201 Created`

```
{
  "id": 101,
  "code": "SO-12345678",
  "status": "Pending",
  "channel": "LINE",
  "total": 748,
  "paid": 0,
  "note": "Rush delivery",
  "items": [
    { "sku": "SHIRT-WHITE-M", "qty": 2, "price": 299 },
    { "sku": "HAT-BLACK",     "qty": 1, "price": 150 }
  ]
}
```

Error Responses

Status	Condition
`401`	API key invalid
`400`	Missing `items`, or an SKU not found
`400`	SKU is archived and cannot be ordered
`400`	`deliver_at` is earlier than the product's lead time
`400`	Missing `customer_id` (only for orgs with it required)
`400`	This `customer_id` was blocked by Waylar staff (a QR-forwarded-to-scam-someone-else prevention measure) — the error message includes the block reason

---

Create a Buyback Order

A buyback order (`order_type: "buy"`) is the shop buying an item back from a customer — the customer is the seller, the shop pays. Unlike a regular (`"sale"`) order, **money flows out of the shop to the customer**, only after someone verifies (approves) it — it's never paid out the moment the order is created.

The money leaves through whichever route Waylar has configured — not something a partner needs to know or integrate against. Only one thing follows from it: the customer's recipient details must be complete (see point 1 below).

Before creating this order, you need:

1. The customer (`customer_id`) must already have a receiving bank account set via `PATCH /customers/{id}` — `bank_account_no` and (`bank_account_name` or `given_name` + `surname`) must both be filled in. An `email` should always be set too — some payout routes require it, and the order cannot be created without it. (**The customer's 13-digit `tax_id` is required too**, and **`address_street` + `address_city`** — the tax id only on some payout routes, the address on all of them.)
2. Enough balance in the **outbound wallet** (order total × 1.002 — see the fee note below). This isn't checked at order-creation time, only at verify time (next step). Only Waylar staff can top up this wallet — there's no endpoint for a partner to do it themselves.
3. If the shop's payout route is INET, the shop (org) must have its own tax id set under **Settings > Payout** in OMS beforehand (once, not per customer — no endpoint for a partner to set this; contact Waylar if it isn't set yet).

Request Body — same as Create Order, plus:

Field	Type	Required	Description
`order_type`	string	✅	Must be `"buy"`
`customer_id`	number	✅	The customer to buy back from — always required, regardless of org settings
`items`	array	✅	Same as Create Order — `items[].price`, if omitted, defaults to the product's cost price, not its selling price

The order's total must be **at least ฿300** (no upper cap) — too small isn't worth the 0.2% payout fee.

Example Request

```
curl -X POST https://oms.waylar.net/api/v1/orders \
  -H "X-API-Key: wlr_abc123..." \
  -H "Content-Type: application/json" \
  -d '{
    "order_type": "buy",
    "customer_id": 52,
    "items": [
      { "sku": "CARD-RARE-001", "qty": 1, "price": 6000 }
    ],
    "note": "Buying back a rare card"
  }'
```

Response `201 Created`

```
{
  "id": 210,
  "code": "BO-51305413",
  "order_type": "buy",
  "status": "Pending",
  "total": 6000,
  "buy_verify_status": "pending"
}
```

Buyback order codes start with `BO-` (regular sale orders use `SO-`). `buy_verify_status` always starts at `"pending"` — no money moves until it's verified (see next section).

Error Responses

Status	Condition
`401`	API key invalid
`400`	Missing `items`, or an SKU not found
`400`	Missing `customer_id`
`400`	`customer_id` not found
`400`	Customer has no receiving bank account set (see PATCH /customers/{id})
`400`	Customer has no 13-digit tax id set (some payout routes only)
`400`	Customer has no email set (some payout routes only)
`400`	Customer has no address set — `address_street` / `address_city`
`400`	Order total is below ฿300

```
{ "error": "customer ยังไม่ได้ตั้งค่าบัญชีรับเงิน — ตั้งค่าก่อนผ่าน PATCH /v1/customers/{id}" }
```

Verify a Buyback Order

```
POST /api/v1/orders/{code}/buy-verify
```

Hard gate before any real payout — every buyback order must go through this endpoint (or be approved from the OMS dashboard) before money actually reaches the customer. There is no other path that pays out without going through this step.

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)
`Content-Type`	`application/json`

Path Parameters

Param	Type	Description
`code`	string	Order code from the create-order response (`BO-...`)

Body

Field	Type	Required	Description
`decision`	string	✅	`"approve"` (pay out for real) or `"reject"` (cancel the order)
`note`	string		Note (stored in the event log)
`receiver_tax_id`	string		13-digit tax ID of the payee (the customer) on the payout voucher
`merchant_tax_id`	string		13-digit tax ID of the merchant
`merchant_name`	string		Merchant name (max 100 characters)

`receiver_tax_id` / `merchant_tax_id` / `merchant_name` are optional — send all three or none (some of them → `400`). Tax IDs may contain dashes; only the digits are used. When none is sent, the customer's tax ID and the merchant's name/tax ID on file are used (unchanged behaviour). Only read on `decision: "approve"`.

Example Request

```
curl -X POST https://oms.waylar.net/api/v1/orders/BO-51305413/buy-verify \
  -H "X-API-Key: wlr_abc123..." \
  -H "Content-Type: application/json" \
  -d '{ "decision": "approve" }'
```

Response `200`

```
{ "ok": true, "buy_verify_status": "verified", "estimated_payout_at": "2026-09-28T13:10:00.000Z" }
```

`estimated_payout_at` (v1.21.0) — roughly when the money actually goes out. `null` when `decision: "reject"` (nothing was paid out). On `"approve"`:

- **Paid via INET** — a real, precise time: the next tick of IC-TWS's batch clock, which runs every 5 minutes on the wall clock (13:05, 13:10, 13:15, ...) — **not** "5 minutes after this call". A voucher accepted at 13:07 still waits for 13:10, whatever time it was actually accepted at.
- **Paid via Xendit** — just Waylar's own rough estimate (~30 min), **not a confirmed SLA from Xendit** — Xendit publishes no fixed batch clock the way IC-TWS does.
- **Paid via MANA** (v1.27.0) — usually settles during this very call; if not, Waylar re-checks its status every 5 minutes, so this is a ~10 min estimate, not an SLA. At most **฿200,000 per order** (v1.32.0; was ฿50,000) — above that you get an error and the order stays `"pending"` (nothing was debited).

Whatever the rail, the `order.buy_payout_succeeded` / `order.buy_payout_failed` webhook **can arrive before this call's response** (the payout confirms faster than the response returns). Treat the webhook, or `GET /orders/{code}`, as the source of truth — never let this response overwrite a status a webhook already gave you.

`decision`:

- **`"approve"`** — checks whether the outbound wallet can cover the order (order total × 1.002); if so, sends the payout to the customer's bank account on file immediately. `buy_verify_status` becomes `"verified"` and `buy_payout_status` becomes `"sent"` — **which is not yet money in their account**: the provider has only accepted the request and can still refuse it afterwards. The order becomes `"Received"` (stock added) only once the provider confirms the transfer (`buy_payout_status: "paid"`). If it is refused later, `buy_verify_status` goes back to `"pending"` with `buy_payout_status: "failed"`, the amount is returned to the wallet automatically, and the order can simply be verified again. — **safe to retry if the previous call returned an insufficient-balance error** (nothing was debited or touched on that failure). **Do not retry if `buy_verify_status` comes back as `"unknown"`** — that means the provider's result was unclear (timeout/5xx) and the money may genuinely have already gone out; wait for Waylar to check with the provider first.
- **`"reject"`** — cancels the order (`status: "Cancelled"`), reversing stock if any had already been received. No money moves. `buy_verify_status` becomes `"rejected"`.

Error Responses

Status	Condition
`400`	`decision` is not `"approve"` / `"reject"`
`401`	API key invalid

Status	Condition
`404`	Order code not found
`409`	Order is not a buyback order
`409`	Order is not awaiting verification (already resolved / not pending)
`409`	Insufficient outbound wallet balance — safe to retry later
`409`	Shop has no tax id set — set it under Settings > Payout, then retry (some payout routes only)
`409`	Customer has no `email` — fill it in, then retry (some payout routes only)
`409`	Order total outside the payable range (฿1–฿2,000,000)

```
{
  "error": "ยอดคงเหลือ (wallet จ่ายเงิน) ไม่พอ ยังไม่จ่าย — ลองใหม่ทีหลังได้",
  "error_code": "INSUFFICIENT_BALANCE",
  "retryable": true
}
```

Every error from this endpoint carries `error_code` and `retryable` (v1.31.0). **Branch on `error_code`** — `error` is Thai text for people and may be reworded. `retryable: true` = calling buy-verify again later with the same data is safe (no money left on this attempt) and may succeed; `false` = fix something first, or do not call again (see the table).

`error_code`	HTTP	`retryable`	Meaning / what to do
`INVALID_REQUEST`	400	false	Bad body (`decision` or voucher fields)
`ORDER_NOT_FOUND`	404	false	No order with this code
`NOT_A_BUYBACK_ORDER`	409	false	Not a buyback order
`ORDER_NOT_AWAITING_VERIFY`	409	false	Already verified/rejected, or not `pending` — read `GET /orders/{code}/status`
`BUSY`	409	true	The wallet is busy with another payout — retry shortly

`error_code`	HTTP	`retryable`	Meaning / what to do
`INSUFFICIENT_BALANCE`	409	true	Outbound wallet too low — top up, then retry
`AMOUNT_OUT_OF_RANGE`	409	false	Amount outside the per-order payout limits
`RECIPIENT_INCOMPLETE`	409	false	Recipient details missing (account / PromptPay / tax id / email) — fix the customer
`INVALID_RECIPIENT`	409	false	Account / PromptPay number malformed
`RECIPIENT_NOT_FOUND`	409	true	The bank says the recipient account does not exist — around 23:00–07:00 (Bangkok) this can be the bank's maintenance; retry after 07:00, and check the account if it persists
`BANK_TIMEOUT`	409	true	Destination bank did not answer; nothing debited
`SERVICE_UNAVAILABLE`	409	true	Payout service temporarily unavailable; nothing debited
`PAYOUT_REJECTED`	409	false	Refused for another reason (see `error`)
`PAYOUT_UNKNOWN`	409	false	**Result unclear, money may have left — do not call again.** Contact Support with the order code
`RESULT_CONFLICT`	409	false	The order changed meanwhile — read its status; do not call again

New codes may be added; for an unknown code, follow the `retryable` it comes with.

Cancel Order

```
POST /api/v1/orders/{code}/cancel
```

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)

Path Parameters

Param	Type	Description
`code`	string	Order code, from the create-order response

Example Request

```
curl -X POST https://oms.waylar.net/api/v1/orders/SO-12345678/cancel \
  -H "X-API-Key: wlr_abc123..."
```

Response `200`

```
{ "order_code": "SO-12345678", "status": "Cancelled" }
```

Cancelling automatically releases any stock already cut for the order. Orders auto-closed because their QR expired go through this same path (see `cancel_reason` in the OMS dashboard).

Error Responses

Status	Condition
`401`	API key invalid
`404`	Order code not found
`409`	Order is already cancelled
`409`	Order has already been paid (use a refund flow instead)

```
{ "error": "order already paid" }
```

Generate Payment QR Code

```
POST /api/v1/orders/{code}/qrcode
```

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)

Path Parameters

Param	Type	Description
`code`	string	Order code, from the create-order response

Example Request

```
curl -X POST https://oms.waylar.net/api/v1/orders/SO-12345678/qrcode \
  -H "X-API-Key: wlr_abc123..."
```

Response `200`

```
{
  "order_code": "SO-12345678",
  "qrcode_base64": "iVBORw0KGgoAAAANSUhEUgAA..."
}
```

`qrcode_base64` is a base64-encoded PNG (no `` prefix) — prepend that prefix yourself to render it directly in an `<img>` tag.

The QR is valid for 30 minutes — calling this endpoint again within 30 minutes of the last call (and before the order is paid) returns the same QR instead of generating a new one from INET.

Error Responses

Status	Condition
`401`	API key invalid
`404`	Order code not found
`409`	Order has already been paid
`429`	The payment gateway is busy or temporarily unavailable — retry after `Retry-After` seconds (nothing was created)
`502`	Payment gateway (INET) did not return a QR

```
{ "error": "order already paid" }
```

Pay an Order by Card

```
POST /api/v1/orders/{code}/card-payment
```

Returns a hosted payment page where the customer pays by **credit/debit card** (no installments yet) and completes 3DS/OTP. Card data never passes through the partner or Waylar (v1.30.0).

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)

Path Parameters

Param	Type	Description
`code`	string	order code from the create-order response

Body (optional, JSON — v1.34.0)

Field	Type	Description
`return_url`	string	where the customer's browser goes after the card page — `https` only, no `user:pass@`, at most 500 characters. Include whatever you need to find the order (e.g. the order code). Omitted = Waylar's default "payment successful" page

Example Request

```
curl -X POST https://oms.waylar.net/api/v1/orders/SO-12345678/card-payment \
  -H "X-API-Key: wlr_abc123..."
```

With a return URL:

```
curl -X POST https://oms.waylar.net/api/v1/orders/SO-12345678/card-payment \
  -H "X-API-Key: wlr_abc123..." \
  -H "Content-Type: application/json" \
  -d '{"return_url":"https://shop.example.com/payment/done?order=SO-12345678"}'
```

Response `200`

```
{
  "order_code": "SO-12345678",
  "payment_url": "https://pay.beamcheckout.com/...",
  "expires_at": "2026-10-16T05:15:00.000Z"
}
```

- `payment_url` — send the customer there (redirect or link). **Do not embed it in an iframe.**
- `expires_at` — the page works until then (about 15 minutes); `null` = the gateway gave no expiry.
- Calling again returns a new page and **switches the previous one off** (only the latest can be paid).
- Requesting a QR (Generate Payment QR Code) afterwards also switches the card page off.
- Once paid it behaves exactly like a QR payment: `GET /orders/{code}/status` shows `paid` and the usual webhooks fire.
- **Arriving at `return_url` is not proof of payment** — the customer also lands there after giving up or failing. Confirm with the webhook or `GET /orders/{code}/status` (`is_paid: true`). `return_url` is not stored: a later call without it uses the default page.

Error Responses

Status	Condition
`400`	`return_url` is not an https URL, carries `user:pass@`, or is longer than 500 characters
`401`	invalid API key
`404`	order code not found
`409`	order already paid · a QR for it can still be paid (wait for it to expire, ~5 min) · card payments are switched off
`422`	the amount is over what the card gateway accepts
`502`	the gateway returned no page, or the previous page could not be switched off — safe to retry

---

Check Payment Status

```
GET /api/v1/orders/{code}/status
```

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)

Example Request

```
curl https://oms.waylar.net/api/v1/orders/SO-12345678/status \
  -H "X-API-Key: wlr_abc123..."
```

Response `200`

```
{
  "order_code": "SO-12345678",
  "payment_status": "paid",
  "paid": true,
  "is_paid": true,
  "paid_amount": 748,
  "paid_at": "2026-08-25T10:12:00.000Z",
  "payment_reference_id": "INET1234567890",
  "note": "ส่งด่วน",
  "settlement_status": "sent",
  "transfer": {
    "paid_at": "2026-08-25T10:12:00.000Z",
    "amount": 748,
    "reference_id": "INET1234567890",
    "payer_name": "Somchai Jaidee",
    "review_status": "",
    "payout": {
      "pv_no": "PVB1a2b3c4d51788534001410",
      "status": "sent",
      "net_amount": 736.03,
      "fee_amount": 11.97
    }
  }
}
```

Response `200` — example while `held` (payer name doesn't match the customer, v1.11.0)

```
{
  "order_code": "SO-12345678",
  "payment_status": "held",
  "paid": false,
  "is_paid": false,
  "paid_amount": 748,
  "paid_at": "2026-08-25T10:12:00.000Z",
  "payment_reference_id": "INET1234567890",
  "note": "ส่งด่วน",
  "settlement_status": null,
  "customer": {
    "id": 123,
    "code": "C-00123",
    "name": "Somying Rakdee",
    "phone": "0812345678",
    "email": "somying@example.com",
    "bank_code": "014",
    "bank_account_no": "******7890",
    "bank_account_name": "Somying Rakdee",
    "bank_account_type": "account"
  },
  "transfer": {
    "paid_at": "2026-08-25T10:12:00.000Z",
    "amount": 748,
    "reference_id": "INET1234567890",
    "payer_name": "Somying Rakdee",
    "review_status": "pending",
    "payout": null
  }
}
```


`paid` is a **boolean** here (different meaning from the number `paid` on `GET /orders`). Use `is_paid` (boolean) and `paid_amount` (number) instead if that ambiguity matters to you — the original `paid` stays for backward compat.

`payment_reference_id` — INET's own transaction reference number, for reconciling against the partner's own bank slip/statement. `null` if unpaid, or paid via a method with no such reference (e.g. COD). Pair with `GET /orders?payment_status=unpaid` to pull a list of not-yet-paid orders directly.

`payment_status` is one of 5 values (v1.11.0 added `held` / `failed` — see Changelog):

Value	Meaning	`paid` / `is_paid`
`unpaid`	No checkout/QR has ever been generated for this order	`false`

Value	Meaning	`paid` / `is_paid`
`pending`	A checkout/QR was generated; waiting on the customer/gateway	`false`
`paid`	Money arrived**and** was released to the partner (no review held it)	`true`
`held`	Money arrived, but the payer's name doesn't match the customer — held pending review (see `transfer.review_status: "pending"`). Not yet released to the partner	`false`
`failed`	Money arrived, but the review was rejected (`transfer.review_status: "rejected"`) — **this does not mean the payment itself failed**. The money is real; it just won't be released to the partner. Refund is handled manually outside this API	`false`

`unpaid` / `pending` have existed since v1.0 — poll this endpoint after showing the customer a QR code to check whether it's been paid. `held` / `failed` are states that used to read as plain `paid` before v1.11.0 (see Changelog for why this changed).

`transfer` groups the same transfer details the OMS screens show. It is `null` **only** while `payment_status` is `unpaid` / `pending` (no money has arrived at all yet) — while `held` / `failed` (money arrived, just not released) `transfer` is still populated normally. Every flat field above is unchanged and stays, so this is additive.

- `transfer.payer_name` — the name on the account the customer paid from, as the gateway reported it. `null` when the gateway sends no payer, or for payments confirmed before this was recorded — always treat it as optional. Only the **name** is stored; the account number both gateways also send is deliberately not kept. As of 2026-09, only **Xendit** is confirmed to send a real payer name — INET is not yet confirmed and will always come back `null` until announced.
- `transfer.review_status` — `""` normally. `"pending"` means the money arrived under a name that doesn't match the order's customer, so the payout is held until someone confirms it in OMS (`transfer.payout` stays `null` meanwhile). Ends as `"approved"` or `"rejected"`. The same state is mirrored at the top level in `payment_status` (`pending` → `held` , `rejected` → `failed`) so a partner reading only the flat fields still sees it correctly.

  > **Note**: this same concept has 3 different key names depending on the endpoint: `transfer.review_status` (here), `review_status` (top-level in the `order.payer_review` / `order.payer_review_resolved` webhooks), and `payer_review_status` (response of `POST /orders/{code}/payer-review`). Same

> possible values everywhere (`""` / `pending` / `approved` / `rejected`), just a different key name per the endpoint it was added to.

- `transfer.payout` — what the partner is owed for this order and where that payout stands. Uses the same enum as `settlement_status` below (`pending` / `sent` / `paid` / `failed`). `pv_no` is the payment voucher to quote to IC-TWS when chasing it. `null` if the order has no settlement row yet (including while `held` — no settlement row is created until the review is approved).

`settlement_status` is the status of the net payout (after platform fee deduction) to the partner via IC-TWS — one of `pending` | `sent` | `paid` | `failed` , or `null` if this order hasn't entered the settlement pipeline yet (not yet paid, or paid but still held in review).

Sale order platform fee — default **1.6%** of the sale amount (example above: 748 × 1.6% ≈ 11.97), but configurable per org by Waylar. No field exposes this rate directly — read it back from `transfer.payout.fee_amount` / `net_amount` on an already-paid order instead.

`customer` — **buyback orders only** (`order_type: "buy"`); always `null` for a sale order, and `null` for a buyback with no customer attached. This is the account the payout goes to, so a failed payout can be reconciled without a second `GET /customers/{id}` call. Always scoped to the API key's own org.

`bank_account_no` is **masked to its last 4 digits**, the same rule the webhooks use. These details are ones the partner supplied in the first place through `PATCH /customers/{id}` , so the full number adds nothing they do not already hold — it would only end up in the logs of everything that polls this endpoint. `tax_id` is left out here for the same reason.

`note` (v1.33.0) — the order's note as created; `null` when empty.

`buy_verify_status` — buyback orders only (`order_type: "buy"`). Always `null` for sale orders. Possible values: `"pending"` (awaiting verify — see Verify a Buyback Order), `"verified"` (paid out successfully), `"rejected"` (rejected, order cancelled), `"unknown"` (the payout provider's result was unclear, waiting on Waylar to check — **do not retry buy-verify while in this state**), or `"processing"` (payout in flight, normally resolves within seconds — contact Waylar if it stays like this unusually long). `buy_verified_at` is when verify completed (`null` if not yet).

`buy_payout_status` — buyback orders only as well. This is what says whether the money actually reached the recipient, which `buy_verify_status` cannot ("verified" only ever means the request was accepted): `null` / `""` (no payout attempted), `"sent"` (sent, awaiting confirmation), `"paid"` (confirmed transferred — this is the point the order becomes `"Received"` and stock is added), `"failed"` (refused after acceptance; no money left, the amount is back in the wallet and `buy_verify_status` has returned to `"pending"`). The reason is in `buy_payout_failure_reason` and the confirmation time in `buy_payout_settled_at` (`null` until confirmed).

Error Responses

Status	Condition
`401`	API key invalid
`404`	Order code not found

Resolve a Payer Review

```
POST /api/v1/orders/{code}/payer-review
```

When `transfer.review_status` is `"pending"` (money arrived, but the payer's name doesn't match the order's customer — see Check Payment Status), this lets the partner decide directly via the API instead of going into the OMS dashboard — the partner usually knows best whether the payer really is their customer.

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)
`Content-Type`	`application/json`

Body

Field	Type	Required	Description
`decision`	string	✅	`"approve"` or `"reject"`
`note`	string		Note (kept in the order's event log)
`reviewed_by`	string		Reviewer's name/email (defaults to `"api"` if omitted)

`decision` means one of two different things depending on the situation (the server figures out which from the order itself):

- **Ordinary payer-name mismatch** — `approve` = it is the same person, pay the merchant normally (schedules a settlement). `reject` = it isn't, holds the payout for a human to sort out.
- **Money arrived after the order was already cancelled** — `approve` = keep the money, pay the merchant normally (the order stays `Cancelled` — stock already released is not restored). `reject` = confirm the money should not be kept — a refund to the customer is needed instead.

Both situations return the exact same response/webhook shape — the example below is the mismatch case; "money after cancellation" gets an identical response (`payer_review_status` / `review_status` use the same values in both cases, there's no special value for this one).

```
curl -X POST https://oms.waylar.net/api/v1/orders/SO-12345678/payer-review \
  -H "X-API-Key: wlr_abc123..." \
  -H "Content-Type: application/json" \
  -d '{ "decision": "approve", "reviewed_by": "staff@merchant.com" }'
```

Response `200` (approve)

```
{ "ok": true, "order_code": "SO-12345678", "payer_review_status": "approved",
"reviewed_at": "2026-09-14T10:00:00.000Z" }
```

Response `200` (reject)

```
{ "ok": true, "order_code": "SO-12345678", "payer_review_status": "rejected",
"reviewed_at": "2026-09-14T10:00:00.000Z" }
```

`approve` fires both `order.paid` and `order.payer_review_resolved` — `order.paid` was never sent while the review was pending (the system guarantees this ordering; there is no path where `order.paid` could already have gone out before a review starts). `reject` fires only `order.payer_review_resolved` — **no** `order.paid` follows. See Outbound webhooks.

What happens after `reject`: closes the review as `"rejected"`, and **the system now automatically opens a support ticket for Waylar's own Support team** (v1.11.0) so they process the customer refund — this doesn't touch this API's contract at all (no new field/response/ webhook for partners), it's purely an internal Waylar mechanism. There is still no refund endpoint or webhook exposed to partners anywhere in this API, and no webhook fires when the refund is actually completed — there is no formal partner-facing refund flow yet.

Orders with no `customer_id`: the payer-name review needs a customer to compare against — if the order has no `customer_id` at all (an org that doesn't require it), this check is **skipped entirely** (never held, no matter what name it was paid under). `transfer.review_status` will always be `""` for such an order — it can never show `"pending"`.

Error Responses

Status	Condition
`400`	`decision` isn't `"approve"` / `"reject"`
`401`	API key invalid
`404`	Order code not found
`409`	Order isn't awaiting a payer-name review
`409`	Someone else already resolved this

Wallet

Check Wallet Balance

```
GET /api/v1/wallet-balance
```

Check your org's own wallet balances before creating/verifying a buyback order — avoids guessing and hitting a `409` on verify.

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)

Example Request

```
curl https://oms.waylar.net/api/v1/wallet-balance \
  -H "X-API-Key: wlr_abc123..."
```

Response `200`

```
{
  "inbound_wallet": 12345.67,
  "outbound_wallet": 890.12
}
```

- `inbound_wallet` — money already earned from sales that Waylar hasn't transferred to your bank account yet (net of platform fee). Not directly spendable on buybacks, but shows how much more could be moved into the outbound wallet.
- `outbound_wallet` — **the balance actually spendable on buyback orders.** Must be ≥ (order total × 1.002) for verify to succeed. Two ways to top it up: ask Waylar staff to transfer from the inbound wallet, or **request a QR and scan-and-pay it yourself** — see Request a Wallet Top-up QR.

Error Responses

Status	Condition
`401`	API key invalid

Request a Wallet Top-up QR

```
POST /api/v1/wallet-topup/qrcode
```

Request a QR to scan-and-pay yourself, topping up the **outbound wallet** directly — this is genuinely new money from outside the platform, unrelated to the inbound wallet's balance (unlike the internal transfer Waylar staff can do on your behalf).

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)

Body

Field	Type	Required	Description
`amount`	number	✓	Amount to top up (THB), must be > 0, at most 2 decimals

Example Request

```
curl -X POST https://oms.waylar.net/api/v1/wallet-topup/qrcode \
  -H "X-API-Key: wlr_abc123..." \
  -H "Content-Type: application/json" \
  -d '{"amount": 5000}'
```

Response `201`

```
{
  "code": "WT-12345678",
  "amount": 5000,
  "qrcode_base64": "iVBORw0KGgoAAAANSUhEUgA..."
}
```

- `code` — this request's own reference number, used to poll Check Wallet Top-up Status
- `qrcode_base64` — the QR as a base64 PNG — gateway (INET/Xendit) is auto-selected the same way a sale order's QR is, nothing to specify
- **the amount actually credited is net of a 0.02% fee** (e.g. topping up ฿5,000 credits ฿4,999 to the wallet) — the same fee Waylar staff apply when recording a slip by hand
- paying it fires a `wallet_topup.paid` webhook too (see Outbound webhooks) — polling this endpoint as well is fine, just not required

Error Responses

Status	Condition
`401`	API key invalid
`400`	Missing `amount`, or `amount` isn't a positive number with at most 2 decimals
`429`	The payment gateway is busy or temporarily unavailable — retry after `Retry-After` seconds
`502`	Failed to issue a QR from the gateway — safe to retry

Check Wallet Top-up Status

```
GET /api/v1/wallet-topup/{code}/status
```

Check whether a top-up request has been paid yet — `code` is the value returned by Request a Wallet Top-up QR.

Headers

Header	Value
`X-API-Key`	`wlr_<key>` (required)

Example Request

```
curl https://oms.waylar.net/api/v1/wallet-topup/WT-12345678/status \
  -H "X-API-Key: wlr_abc123..."
```

Response `200`

```
{
  "code": "WT-12345678",
  "amount": 5000,
  "status": "paid",
  "paid_at": "2026-09-26T10:15:00.000Z",
  "created_at": "2026-09-26T10:12:30.000Z"
}
```

- `status` — `pending` (not yet paid) | `paid` (paid, credited to the outbound wallet) | `expired` (no payment seen for 7 days after the request was created). An `expired` request may still become `paid` if the gateway confirms the payment later — the wallet is credited and `wallet_topup.paid` fires as usual — so treat `expired` as "not paid so far", not as final.
- `amount` — the full amount the QR asked for (**before** the 0.02% fee — the actual amount credited to the wallet is slightly less)

Error Responses

Status	Condition
`401`	API key invalid
`404`	`code` not found (or belongs to a different org)

Support Tickets

Report an Order Problem

Report a problem on a specific order to Waylar's Support team — the same feature as "Report a problem" in the OMS dashboard itself, just callable via API instead of a staff member clicking through the web page. **One order can only have one open ticket at a time** — it has to be `closed` before a new one can be opened.

Attach a file (optional) — upload it first, then pass its url when opening the ticket

```
POST /api/v1/orders/{code}/support/upload
```

Header	Value
`X-API-Key`	`wlr_<key>` (required)

`multipart/form-data`, field name `file` — one file per request, PDF or image (`png` / `jpg` / `webp`) only, up to 16 MB.

```
curl -X POST https://oms.waylar.net/api/v1/orders/SO-12345678/support/upload \
  -H "X-API-Key: wlr_abc123..." \
  -F "file=@slip.jpg"
```

Response `200`

```
{ "url": "support-tickets/1234/9f3a1b2c...jpg", "view_url":
"https://storage.googleapis.com/...", "filename": "slip.jpg" }
```

Pass the `url` value (not `view_url`) into `file_urls` when opening the ticket below — `view_url` is only a 10-minute signed URL for viewing the file right away; it expires and doesn't need to be kept.

Open a new ticket

```
POST /api/v1/orders/{code}/support
```

Header	Value
`X-API-Key`	`wlr_<key>` (required)

Body

Field	Type	Required	Description
`message`	string	✅	Description of the problem
`file_urls`	string[]		`url` values from the upload above (optional)
`reported_by_email`	string	✅	Reporter's email — Support's replies/status updates are emailed here
`reported_by_name`	string		Reporter's name (optional)

An API key carries no identity for a specific staff member (unlike logging into the web dashboard), so `reported_by_email` has to be supplied on every call.

```
curl -X POST https://oms.waylar.net/api/v1/orders/SO-12345678/support \
  -H "X-API-Key: wlr_abc123..." \
  -H "Content-Type: application/json" \
  -d '{
    "message": "Customer scanned the QR and it showed a different bank account than
ours",
    "file_urls": ["support-tickets/1234/9f3a1b2c...jpg"],
    "reported_by_email": "staff@merchant.com",
    "reported_by_name": "Merchant A staff"
  }'
```

Response `201`

```
{ "ok": true, "ticket_id": 42, "status": "received" }
```

Error Responses

Status	Condition
`400`	`message` or `reported_by_email` was not sent
`401`	API key invalid

Status	Condition
`404`	Order code not found
`409`	This order already has an open ticket

```
{ "error": "order นี้มีการรายงานปัญหาที่ยังไม่ปิดอยู่แล้ว", "ticket_id": 40 }
```

Check status / timeline

```
GET /api/v1/orders/{code}/support
```

Header	Value
`X-API-Key`	`wlr_<key>` (required)

Query Parameters

Param	Type	Description
`ticket_id`	integer	View a specific past ticket instead of the latest one (optional)

```
curl https://oms.waylar.net/api/v1/orders/SO-12345678/support \
  -H "X-API-Key: wlr_abc123..."
```

Response `200`

```
{
  "ticket": {
    "id": 42,
    "order_code": "SO-12345678",
    "status": "in_progress",
    "reported_by_email": "staff@merchant.com",
    "reported_by_name": "Merchant A staff",
    "created_at": "2026-09-14T03:10:00.000Z",
    "updated_at": "2026-09-14T05:00:00.000Z"
  },
  "events": [
    {
      "id": 100, "type": "opened", "status": null,
      "remark": "Customer scanned the QR and it showed a different bank account than
ours",
      "actor": "", "created_at": "2026-09-14T03:10:00.000Z",
      "files": [{ "url": "support-tickets/1234/9f3a1b2c...jpg", "name": null, "view_url":
"https://storage.googleapis.com/..." }]
    },
    {
      "id": 101, "type": "status_changed", "status": "in_progress",
      "remark": "Checking with the payments team", "actor": "Waylar Support",
"created_at": "2026-09-14T05:00:00.000Z",
      "files": []
    }
  ],
  "history": []
}
```


`ticket` is `null` (with `events` / `history` as empty arrays) if this order has never had a ticket opened.

A ticket's `status` moves `received` → `in_progress` → `resolved` → `closed` — changed only by the Support team's side (this external API has no endpoint to change it yourself). Every change also fires a `support_ticket.status_changed` webhook (see Outbound webhooks) — you don't have to poll this endpoint hard to keep up.

"One open ticket at a time" only clears on `closed` — `resolved` still counts as open; you cannot open a new ticket until Support actually sets it to `closed` (not just `resolved`).

`events[].type` is one of `opened` | `customer_note` | `status_changed` — only `status_changed` carries a non-null `status` field.

`history` lists this order's earlier tickets (if it has had more than one) — each entry is a short summary only, not the full timeline. Call again with `?ticket_id=<id>` for that ticket's full timeline.

Error Responses

Status	Condition
`401`	API key invalid

Status	Condition
`404`	Order code not found

Check Ticket Status (by ticket_id)

A lean status check for a single ticket, keyed directly by `ticket_id` — for when that's all you have (e.g. from a `support_ticket.status_changed` webhook) and you don't want the full timeline that `GET /orders/{code}/support` above returns.

```
GET /api/v1/support-tickets/{ticket_id}/status
```

Header	Value
`X-API-Key`	`wlr_<key>` (required)

```
curl https://oms.waylar.net/api/v1/support-tickets/42/status \
  -H "X-API-Key: wlr_abc123..."
```

Response `200`

```
{
  "ticket_id": 42,
  "order_code": "SO-12345678",
  "status": "in_progress",
  "created_at": "2026-09-14T03:10:00.000Z",
  "updated_at": "2026-09-14T05:00:00.000Z"
}
```

Error Responses

Status	Condition
`401`	API key invalid
`404`	Ticket not found (or belongs to another org)

API Keys

Creating and revoking an API Key is only done from **Settings** → **API Keys** in the OMS dashboard — not an endpoint of this external API (a key authenticates *into* this API; it isn't something the external API manages about itself).

Notes

- An API key is scoped to one org — products can only be created in the org that issued the key.
- SKUs must be unique per org across the whole system (not just per key).
- `margin` in responses is computed as `(price - cost) / price * 100` (%).

Outbound webhooks

Set the URL and secret per org under OMS → Settings → Webhook — **one tab per destination**: the "Order paid" tab covers all `order.*` / `wallet_topup.*` events, the "Support Ticket" tab covers only `support_ticket.status_changed` (v1.20.0). They can point at two different URLs/secrets, or the same one — your choice. Every request is signed with an HMAC-SHA256 of the raw JSON body (before parsing) using that tab's own secret, encoded as **hex**, sent as `x-waylar-signature`.

- **Timestamp/nonce**: none — the payload carries no replay-protection field of any kind.
- **Retry**: best-effort, up to **2 attempts** total (1 initial + 1 retry). No further retries, no durable queue — if a partner endpoint is down or slow for both attempts, that event is lost.
- **Expected response**: any HTTP 2xx is treated as success; anything else is a failure (retried if a retry is still available). The response body is never inspected.
- **Idempotency**: no idempotency key is sent — dedupe on your side if needed (e.g. a `so` + `event` pair against events you've already processed).

Order (`order.*` , `wallet_topup.*`) — configured on the "Order paid" tab

`order.paid` (original payload — no `event` key)

```
{ "so": "SO-12345678", "ref1": "INET1234567890", "amount": 748, "paid_at": "2026-08-25T10:12:00.000Z", "payer_name": "Somchai Jaidee" }
```

- `so` — the order code (same as `code` in `POST /orders`'s response)
- `ref1` — the same payment_reference_id seen on `GET /orders/{code}/status` (omitted/absent if the gateway sent no reference)
- `amount` — the amount actually paid (THB)
- `paid_at` — when the payment was confirmed, ISO 8601 + `Z`
- `payer_name` — new (v1.10.0). The name on the account the customer paid from, `null` if the gateway didn't report one. The existing `so` / `ref1` / `amount` / `paid_at` keys are unchanged — an integration that only reads those is unaffected.

`order.payer_review` — paid, but the payer's name doesn't match (v1.6.0)

Sent **instead of** `order.paid`, not in addition to it. While a payment is held for review you will **not** receive `order.paid`, because the money has not been released to the merchant — telling you "paid" here would have you ship against funds still on hold.

`order.paid` arrives later, when staff approve it. An integration that only listens for `order.paid` therefore still works end to end without knowing about these events at all; it simply hears about the order later in the cases that need a review.

```
{
  "event": "order.payer_review",
  "so": "SO-12345678",
  "ref1": "INET1234567890",
  "amount": 748,
  "paid_at": "2026-08-25T10:12:00.000Z",
  "payer_name": "Somying Rakdee",
  "customer_name": "Somchai Jaidee",
  "review_status": "pending"
}
```

`order.payer_review_resolved` — the review ended (v1.6.0)

```
{
  "event": "order.payer_review_resolved",
  "so": "SO-12345678",
  "ref1": "INET1234567890",
  "amount": 748,
  "paid_at": "2026-08-25T10:12:00.000Z",
  "review_status": "approved"
}
```

`review_status` is `"approved"` (the payout joins the next run) or `"rejected"` (it stays on hold and is handled by hand).

`order.buy_payout_failed` — a buyback payout failed (v1.14.0)

Buyback orders (`BO-`) only. Sent when the payout rail accepted the request and then refused it — **no money left the account**.

```
{
  "event": "order.buy_payout_failed",
  "so": "BO-23101435",
  "amount": 10000,
  "payout_status": "failed",
  "failure_reason": "TRANSFER_ERROR — destination rejected the transfer",
  "error_code": "PAYOUT_REJECTED",
  "payout_ref": "BO-23101435",
  "refunded": true,
  "retryable": true,
  "bank_account_name": "Lertchai Naewsoon",
  "bank_account_no": "******5608",
  "bank_code": "014"
}
```

- `error_code` (v1.31.0) — the same codes as buy-verify's table; read this instead of `failure_reason`. This event's own `retryable: true` only means the order is `pending` again and can be re-approved; whether that can succeed with the same data is the table's `retryable` for the code (e.g. `INVALID_RECIPIENT`: fix the account first).
- `refunded: true` — the amount is already back in the org's outbound wallet. **Do not ask for a refund.**
- `retryable: true` — `buy_verify_status` is back to `"pending"`, so `POST /orders/{code}/buy-verify` can be called again once the underlying problem (usually the recipient's bank details) is fixed.
- `bank_account_no` is **masked to its last 4 digits**. You supplied this number when creating the customer, so it is not sent back in full; `customer` on `GET /orders/{code}/status` masks it the same way. The full number is on your own side, where it came from.
- The order does **not** become `"Received"` and no stock moves (since v1.13.0).

`order.buy_payout_succeeded` — a buyback payout completed (v1.14.0)

Sent when the rail **confirms** the transfer, not when it accepts it. This is also the point the order becomes `"Received"` and stock moves.

```
{
  "event": "order.buy_payout_succeeded",
  "so": "BO-23101435",
  "amount": 10000,
  "payout_status": "paid",
  "payout_ref": "BO-23101435",
  "settled_at": "2026-09-24T04:35:10.000Z"
}
```

> Nothing is sent between a successful verify (`buy_payout_status: "sent"`) and one of these two events. If a payout stays quiet for an unusually long time, poll `GET /orders/{code}/status` or contact Waylar.

`wallet_topup.paid` — a wallet top-up QR was paid (v1.18.0)

Sent when the QR requested from `POST /wallet-topup/qrcode` is paid — unrelated to any order, so there's no `so` field.

```
{
  "event": "wallet_topup.paid",
  "code": "WT-12345678",
  "gross_amount": 5000,
  "fee_amount": 1,
  "net_amount": 4999,
  "paid_at": "2026-09-27T03:15:00.000Z"
}
```

- `code` — same value returned by `POST /wallet-topup/qrcode`, also usable with `GET /wallet-topup/{code}/status`
- `net_amount` — the amount actually credited to the outbound wallet (net of the 0.02% fee), different from `gross_amount` (the QR's full requested amount)

Event sequence

Situation	Events, in order
Normal payment	`order.paid`
Payer mismatch → approved	`order.payer_review` → `order.paid` → `order.payer_review_resolved`
Payer mismatch → rejected	`order.payer_review` → `order.payer_review_resolved` (**no** `order.paid`)
Paid after cancellation	`order.payer_review` → (on approval) `order.paid` → `order.payer_review_resolved`
Buyback: verified → rail confirms	`order.buy_payout_succeeded`
Buyback: verified → rail refuses	`order.buy_payout_failed` (already refunded, can be verified again)
Wallet top-up QR paid	`wallet_topup.paid`

These 6 events in this group are all there is — no webhook for anything else order-related. Poll `GET /orders/{code}/status` (or `GET /wallet-topup/{code}/status` for a top-up) if you need to know about:

- An order being cancelled (both an explicit `POST /orders/{code}/cancel` and auto-expiry when a QR times out)
- `settlement_status` changing (`pending` → `sent` → `paid` / `failed`)
- A buyback order's verify result the moment it is verified (`buy_verify_status` → `"verified"`) — the two events above fire when the **rail confirms the outcome**, not when the request is first accepted

---

Support Ticket (`support_ticket.*`) — configured on the "Support Ticket" tab

A different URL/secret from the order group above (its own tab) — same retry/signature/idempotency mechanics, described at the top of this page.

`support_ticket.status_changed` — a support ticket's status changed (v1.19.0)

Sent every time Waylar Support changes the status of a ticket opened via `POST /orders/{code}/support` (including one the system opened on its own). **`event` is always distinct from every event in the order group** (`support_ticket.status_changed` vs. `order.*` / `wallet_topup.*`) — by design, so a partner can branch on `event` first and this can never be mistaken for an order webhook. Since v1.20.0 it doesn't even share a destination — a separate URL/secret entirely.

```
{
  "event": "support_ticket.status_changed",
  "so": "SO-12345678",
  "ticket_id": 42,
  "status": "in_progress",
  "remark": "Checking with the payments team"
}
```

- `so` — the order code this ticket belongs to (pairs with `GET /orders/{code}/support`)
- `status` is one of `received` | `in_progress` | `resolved` | `closed` , same values as `GET /orders/{code}/support`
- `remark` — Support's note on this status change (`null` if none was given)

This group has just the one event — Support adding a `customer_note` to a ticket's timeline (not a `status` change) does **not** fire a webhook — poll `GET /orders/{code}/support` if you need to know about those.

Changelog

v1.34.0

- **`return_url` on `POST /orders/{code}/card-payment`** — optional; after the card page the customer is sent to your https URL instead of Waylar's default page. Not a payment confirmation. Without it nothing changes.

v1.33.0

- **`note`** on every row of `GET /orders` and on `GET /orders/{code}/status` (`null` when empty).
- **`GET /orders?sort=paid_at`** — paid orders by payment time, paged with `cursor` / `next_cursor`. Without `sort` / `cursor` the list is unchanged.

v1.32.0

- **Buyback payout limit raised to ฿200,000 per order** (was ฿50,000) — above it, 409 `AMOUNT_OUT_OF_RANGE`.

v1.31.0

- **`error_code` + `retryable` on `POST /orders/{code}/buy-verify` errors**, and `error_code` on the `order.buy_payout_failed` webhook — branch on the code, not the Thai text. See the code list. `error` / `failure_reason` are still sent as before; existing integrations keep working.

v1.30.0

- **New: `POST /orders/{code}/card-payment`** — pay by credit/debit card on the gateway's hosted page (no installments yet). See Pay an Order by Card.

v1.29.0

- **Reopened `POST /wallet-topup/qrcode` and `GET /wallet-topup/{code}/status`** — they work exactly as before v1.28.0 (same request, response and `wallet_topup.paid` webhook).

v1.28.0

- **Closed `POST /wallet-topup/qrcode` and `GET /wallet-topup/{code}/status`** — both answered `410 Gone` (reopened in v1.29.0).

v1.27.0

- **New: optional `receiver_tax_id`, `merchant_tax_id`, `merchant_name` in the `POST /orders/{code}/buy-verify` body** — name the payee and the merchant on the payout voucher per order. All three or none; none = the customer's and merchant's records on file, as before.
- **New: MANA buyback payout rail** — `buy-verify` request and response unchanged; only `estimated_payout_at` (~10 min) and a ฿50,000-per-order limit differ. An `order.buy_payout_*` webhook can arrive before the `buy-verify` response (every rail).

v1.26.0

- **`address_street` + `address_city` required for buyback orders, on every payout route** (corrected from "some payout routes only" moments after first shipping this) — Xendit's recipient object uses both directly (`city` / `street_line_1`); INET's voucher payload doesn't use them today, but a real address

on file for the recipient is wanted unconditionally. `POST /orders` (`order_type: "buy"`) returns `400` if a customer is missing either. See Create a Buyback Order.

v1.25.0

- **New: `POST /customers/{id}/documents`** — upload a bank book photo (`bank_book`) or an ID card copy (`id_copy`) for a customer, supporting documents for a buyback order's payout bank account. See Upload Customer Document — `PATCH /customers/{id}` and `POST /customers` now include `bank_book_url` / `id_copy_url` in the response (internal paths, not directly browsable links).

v1.24.0

- **Customer `tax_id` is validated** on `POST /customers` and `PATCH /customers/{id}`: it must be 13 digits with a correct check digit, and **may not already belong to another customer in the same org** (`400`). Stored digits-only. See Update Customer Bank Account.

v1.23.0

- **Reverts v1.22.0: buyback payouts over INET use the customer's tax id again**, not the merchant's. The customer's `tax_id` (`PATCH /customers/{id}`) **affects buyback orders again**, required as it was before v1.22.0 — `POST /orders` and `POST /orders/{code}/buy-verify` return a customer-level error rather than a shop-level one when it is missing. A shop no longer needs its own tax id for buyback orders (it still does for ordinary settlement transfers, which is unrelated).

v1.22.0

- **Buyback payouts over INET now use the merchant's own tax id, not the customer's** — IC-TWS always files a payment voucher under the shop's registered tax id, never the recipient's. The customer's `tax_id` (`PATCH /customers/{id}`) **no longer affects buyback orders at all** — it's kept for whatever else you use it for. Shops on the INET payout route must have their own tax id set under Settings > Payout — `POST /orders` (`order_type: "buy"`) and `POST /orders/{code}/buy-verify` now return a shop-level error instead of a customer-level one when it's missing.

v1.21.0

- **New: `estimated_payout_at` field on `POST /orders/{code}/buy-verify`'s response** (`decision: "approve"` only) — roughly when the money goes out: precise for INET (rounds up to the next batch-clock tick), a rough ~30 min guess for Xendit (not a confirmed SLA)

v1.20.0

- **New: `GET /support-tickets/{ticket_id}/status`** — a lean status check keyed directly by `ticket_id`, instead of the full timeline `GET /orders/{code}/support` returns, for when that's all you have (e.g. from a `support_ticket.status_changed` webhook)

- **Split the `support_ticket.status_changed` webhook onto its own URL/secret** — now configurable on its own tab under Settings → Webhook, separate from `order.*` / `wallet_topup.*`
- **Reorganized: split `## Outbound webhooks` into two subsections, "Order" and "Support Ticket"** — events, tables and URL config are grouped by destination now instead of one flat list (the top-level heading is unchanged, existing links still work)

v1.19.0

- **New: `support_ticket.status_changed` webhook** — Support changing the status of a ticket opened via `POST /orders/{code}/support` now fires a webhook too, instead of requiring a poll of `GET /orders/{code}/support` (still fine to poll too, just no longer required). `event` is always distinct from `order.*` / `wallet_topup.*` so it can never be mistaken for an order webhook. See Outbound webhooks.
- **Reorganized: "Report an Order Problem" moved out into its own `## Support Tickets` section, separate from `## Orders`** — the endpoints themselves are unchanged (still `POST` / `GET /orders/{code}/support`), this is a documentation reorganization only.

v1.18.0

- **New: `wallet_topup.paid` webhook** — paying a top-up QR (v1.17.0) now fires a webhook too, instead of requiring a poll of `GET /wallet-topup/{code}/status` (still fine to poll too, just no longer required). See Outbound webhooks.

v1.17.0

- **New: `POST /wallet-topup/qrcode`** — request a QR to scan-and-pay yourself, topping up the outbound wallet directly instead of waiting on Waylar staff to transfer it one request at a time — same 0.02% fee as staff recording a slip by hand. See Request a Wallet Top-up QR.
- **New: `GET /wallet-topup/{code}/status`** — check a top-up request's status (no webhook fires on payment, poll this instead). See Check Wallet Top-up Status.

v1.16.0

- **Calling `buy-verify` again after a failed payout now actually does something.** The order was already sent back to `pending` and reported as `retryable: true`, but the key sent to the payout provider was fixed to the order code, so re-approving read as the same request: one route replayed its earlier answer without moving money, another refused it as a duplicate and dropped straight back to `pending`, forever. Each approval is now its own payout.
- **A late result from an earlier attempt can no longer overwrite the current one** — a delayed or redelivered outcome from a previous payout no longer refunds or re-opens the attempt in flight.
- No field or endpoint changed; existing integrations need no work.

v1.15.0

- **`customer` on `GET /orders/{code}/status` is narrower** — returned for buyback orders only (`null` for a sale order), `bank_account_no` is **masked to its last 4 digits** as it is in the webhooks, and `tax_id` is no longer returned. These are details the partner supplied in the first place; returning the full number on every poll only spread it into more logs.
- **A customer `email` is required for buyback orders**, the same way `tax_id` is (on payout routes that need it) — `POST /orders` returns `400` and `buy-verify` returns `409` without one.

v1.14.0

- **Two new webhooks for buyback orders**: `order.buy_payout_failed` and `order.buy_payout_succeeded`. The money-out side had no webhooks at all before this — the only way to learn a payout's outcome was polling. Existing receivers are unaffected: they keep reading the keys they already know.
- **`customer`** added to `GET /orders/{code}/status`, **for buyback orders only** (`null` for a sale order), so a buyback's destination account can be read without a second `GET /customers/{id}` call. `bank_account_no` is masked to its last 4 digits and `tax_id` is not returned — the same rule the webhooks already follow.

v1.13.0

- **A buyback order only becomes `"Received"` once the provider confirms the money arrived.** Previously it flipped the moment verify succeeded, which meant stock was added while the payout had only been accepted and could still be refused.
  - `buy_verify_status: "verified"` now pairs with `buy_payout_status: "sent"` — sent, awaiting confirmation
  - confirmed → `"paid"`, and the order becomes `"Received"`; stock is added at this point
  - refused later → `"failed"`, `buy_verify_status` returns to `"pending"` and the amount goes back to the wallet automatically — just verify it again, no need to contact anyone
- **`buy_payout_status` / `buy_payout_failure_reason` / `buy_payout_settled_at`** added to `GET /orders/{code}/status` (`null` for sale orders; no existing field changed)

v1.12.0

- **`tax_id` added to `PATCH /customers/{id}`** (and to its response) — the 13-digit tax id of the customer being paid in a buyback order. **Always set it**: the payout route Waylar uses can change at any time and some routes require it. See Update Customer Bank Account
  - `POST /orders` (`order_type: "buy"`) returns `400` if the customer has no 13-digit `tax_id` (only while the route in use requires one)
  - `POST /orders/{code}/buy-verify` gains 2 `409` cases (no `tax_id`, or an order total outside ฿1–฿2,000,000). See Verify a Buyback Order
  - Every existing field and endpoint is unchanged — existing integrations behave exactly as before

v1.11.0

- **`payment_status` gains 2 new values on `GET /orders/{code}/status`: `"held"` and `"failed"`** — both previously read as plain `"paid"` (with `paid` / `is_paid: true`) even though the money had not actually been released to the partner. A partner polling this endpoint instead of listening for webhooks could read that as final and act on it (e.g. ship stock) before a human had confirmed the payer — confirmed live for a real order. Now:
  - `"held"` = money arrived, but the payer's name doesn't match the customer, pending review (`transfer.review_status: "pending"`)
  - `"failed"` = money arrived, but the review was rejected (`transfer.review_status: "rejected"`) — **does not mean the payment itself failed**
  - `paid` / `is_paid` are `false` in both cases (previously `true`)
  - `transfer` is still populated as normal during `held` / `failed` (not `null`) — see Check Payment Status
  - **Behavioral breaking change** for any integration that checks `payment_status === "paid"` as an exact string match rather than the `is_paid` boolean — switch to checking `is_paid`, or add explicit handling for `"held"` / `"failed"`.
- **`reject` on `POST /orders/{code}/payer-review` now automatically opens a support ticket for Waylar's own Support team** — doesn't touch this API's contract at all (no new field/response/ webhook for partners), purely an internal mechanism so Waylar sees a case that needs a refund faster instead of relying on someone to notice it manually — see Resolve a Payer Review
- **`POST /orders` now blocks a `customer_id` that Waylar staff has blacklisted** — returns `400` with the block reason in the error message instead of creating the order/issuing a QR. This is for customers who forward a real payment QR to scam a third party (a real case was confirmed), and is separate from the org's own `blacklisted` flag set in the dashboard (that one is just a pre-shipping warning and never blocks order creation) — see Create Order

v1.10.0

- **`payer_name` added to the `order.paid` webhook** — existing keys (`so` / `ref1` / `amount` / `paid_at`) unchanged. The name on the account the customer paid from, `null` if the gateway didn't report one. See Outbound webhooks.

v1.9.0

- **New Buyback (รับซื้อ) API** — lets a partner buy items back from a customer and get paid out for real, after verify:
  - `order_type: "buy"` added to `POST /orders` — see Create a Buyback Order
  - New `PATCH /customers/{id}` — sets a customer's receiving bank account (see Update Customer Bank Account)
  - New `POST /orders/{code}/buy-verify` — approve/reject a buyback order (hard gate before any real payout, see Verify a Buyback Order)

  - New `GET /wallet-balance` — check your org's own inbound/outbound wallet balances
  - A buyback order's total must be at least ฿300, no upper cap (400 otherwise)
  - `buy_verify_status` / `buy_verified_at` added to `GET /orders/{code}/status` (`null` for sale orders, no impact on existing fields)
  - All existing fields/endpoints are unchanged — existing integrations (implicit `order_type: "sale"`) keep working exactly as before

v1.8.0

- **`POST /orders/{code}/payer-review`** — new. Approve/reject a payer-name review yourself via the API instead of going into the OMS dashboard.
- **The payer-name check is now actually running** (previously off because no gateway was reporting a payer name — some gateways now do). `transfer.review_status` in `GET /orders/{code}/status` can now genuinely show `"pending"` for an order paid through a gateway that sends a payer name (gateways that don't yet still always read `""`).
- **`customer_id`** on `POST /orders` is now conditionally required — only for orgs Waylar has turned it on for (always announced to that org beforehand). Most orgs remain optional; not a breaking change for existing integrations right away.

v1.7.0

- **Order support tickets** — 3 new endpoints: `POST /orders/{code}/support/upload`, `POST /orders/{code}/support`, `GET /orders/{code}/support` let you report a problem on an order directly via the API, instead of a staff member having to go into the OMS dashboard (one order can only have one open ticket at a time).

v1.6.0

- **`transfer.review_status`** added to the `transfer` object (`transfer` itself already existed since v1.5.0 — this release only added a field inside it, not `transfer` itself).
- **Two new webhooks** — `order.payer_review` and `order.payer_review_resolved`. The existing `order.paid` payload is unchanged.
- 🚧 **Not yet active** — the payer-name check and the `customer_id` requirement are both switched off for now (the gateways are not reporting a payer name yet). `customer_id` remains optional, and every successful payment still produces `order.paid` as before. This will be announced before it is turned on.

v1.5.0

- **`transfer`** added to `GET /orders/{code}/status` — the transfer details the OMS screens show, grouped into one object: `paid_at`, `amount`, `reference_id`, `payer_name`, and a `payout` block (`pv_no` to quote to IC-TWS, settlement status, net and fee amounts). `null` until the order is paid.

- **`payer_name`** added to `GET /orders` as well — the name on the account the customer paid from, as the gateway reported it (`{ "name": "..." }`). Only the name is stored, never the account number. `null` when unpaid, when the gateway sends no payer, or for payments confirmed before this was recorded.
- All existing fields are unchanged — purely additive, so current integrations keep working.

v1.4.0

- **`payment_reference_id`** added to `GET /orders` and `GET /orders/{code}/status` — INET's own transaction reference number, for reconciling against the partner's own bank slip/statement (`null` if unpaid or no reference applies).

v1.3.0

- **`POST /orders/{code}/cancel`** added — cancels the order and releases any cut stock automatically.
- Orders auto-cancelled after sitting Pending/unpaid for over an hour now also **clear their QR code** (it used to stick around after cancellation).

v1.2.0

- **`paid_amount`** / **`is_paid`** added to `GET /orders` and `GET /orders/{code}/status` — same value as the existing `paid`, but with an unambiguous type (`paid_amount` is always a number, `is_paid` is always a boolean). The original `paid` stays for backward compat.
- **Unpaid orders now auto-expire** — a sale order left Pending/unpaid for over an hour is automatically cancelled (releasing any cut stock), fixing orders that never closed themselves out after the QR expired.
- **Safer error responses** — `POST /products`, `/customers`, and `/orders` no longer leak raw SQL, bound parameters, or org UUIDs in error responses on a DB failure; just a short, safe message.
- **`weightKg` correctly handles blank input** — `""`, `" "`, `null`, and `undefined` all map to `null` before hitting the DB (a whitespace string previously fell through to `0`).
- **`created_at` is consistently ISO 8601 with `Z`** in `GET/POST /products` and `GET /orders` (previously some of these returned Postgres's raw text format, with no `T` or timezone marker).

v1.1.0

- **New Customer API** — `GET /api/v1/customers` (list) and `POST /api/v1/customers` (create). See the Customers section above for details.
- **QR code caching (30 min)** — `POST /api/v1/orders/{code}/qrcode` now reuses the same QR if called again within 30 minutes (and the order isn't paid yet), instead of generating a fresh one from INET every time.
- **Dynamic pricing on sale orders** — the OMS dashboard's "New order" (sale) screen can now set a per-line price at order-creation time instead of always using the product's list price, via the `items[].price` override that `POST /api/v1/orders` already accepted — no change to the external API contract.